Laboratorio 4 sobre PHP FPM y proxy inverso con Nginx

Informe práctico con pruebas y capturas en Ubuntu y Docker

Tecnología Web I | SIS 0150 | Universidad Privada Domingo Savio

Estudiante: Ilton Said Sandoval Z.

Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty
Fecha: 13 de octubre de 2026
Entorno: Docker Desktop sobre Windows 11 con Ubuntu 24.04
Modalidad: individual

Objetivo

Convertir el portal estático en una aplicación PHP con sesiones y conectar una API HTTP detrás de Nginx. Comprobar los riesgos de publicar código, los atributos de cookies, el control de acceso y los errores 502.

Resultados principales

La instalación inicial con TODO obtuvo 3 de 16 y permitió descargar código PHP. Después de completar los diez TODO, el verificador original obtuvo 16 de 16. Se comprobaron el inicio y cierre de sesión del navegador, la regeneración del identificador, el socket PHP y dos caídas controladas con respuesta 502.

Entorno y evidencias

Se utilizó Ubuntu 24.04 en Docker Desktop sobre Windows 11, con Nginx 1.24.0 y PHP 8.3.6. El único puerto publicado fue 127.0.0.1:8080 hacia el 80 del contenedor. La API escuchó exclusivamente en 127.0.0.1:8081 dentro de Ubuntu. Se usó la cuenta didáctica mamani.rosa con la contraseña Horizonte2026.

Supervisor administró los procesos reales mediante un adaptador local de systemctl. Se completó y validó también la unidad systemd de la hoja. Ubuntu en este contenedor no ejecutó systemd como PID 1; por ello las comprobaciones de actividad y habilitación usaron el adaptador. La API se ejecutó como usuario y grupo www-data, con NoNewPrivileges y una espera de dos segundos antes del reinicio por fallo.

Las figuras son capturas de las páginas reales y de un visor local con las salidas obtenidas. Los atributos de la cookie se consultaron en la interfaz de depuración del navegador; la evidencia de cookies no es una captura del panel Application de DevTools.

Parte A Riesgo de la configuración incompleta

```
cd /lab/lab04
bash instalar_portal.sh
curl -i -H "Host: portal.horizonte.internal" http://127.0.0.1/app/datos.php
bash verificar_lab04.sh
```

Esta prueba se hizo primero, antes de aplicar la configuración corregida. Nginx tenía root en /srv/horizonte/portal y todavía no tenía la ubicación FastCGI. La petición devolvió HTTP/1.1 200 OK y Content-Type: application/octet-stream. El cuerpo contenía el código fuente de datos.php, incluidos los datos de prueba y un hash bcrypt.

Registro de ejecución 04-A-fuga-resumen

Ubuntu 24.04 en Docker Desktop · salida real de los comandos · 13/10/2026

Extracto del registro original 04-A-fuga

```
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Content-Type: application/octet-stream
Content-Length: 756

<?php
// app/datos.php — Datos de ejemplo (en Tecnología Web II vendrán de una base de datos
con PDO).
declare(strict_types=1);

// Contraseñas guardadas con password_hash (bcrypt), NUNCA en texto plano.
// Usuario de prueba: mamani.rosa / Horizonte2026
const PADRES = [
    'mamani.rosa' => [
        'nombre' => 'Rosa Mamani',
        'hash'   => '$2y$10$f2XF0dshC.eMrnbYfEj99eWl6Lc1LeNf7FsEJIKX7SPH60IbIBK1C',
        'hijo'   => 'Lucas Mamani — 5.º de secundaria',
    ],
];
```

Figura 1. Extracto de la descarga del código PHP con Content-Type application/octet-stream

Un atacante podría leer la lógica y los datos publicados, obtener hashes para intentar adivinar contraseñas fuera del servidor y encontrar secretos si el programa los incluyera. Este archivo didáctico incluso contiene la cuenta y contraseña de prueba en un comentario. El fallo demuestra que instalar PHP-FPM no basta: Nginx debe enviarle los archivos PHP y publicar únicamente public/.

Parte A Verificación inicial y acceso privado corregido



Figura 2. Verificador original con 3 de 16 antes de completar los TODO

Pasaron PHP-FPM activo, nginx -t y la prueba de que /index.php no contenía la cadena <?php. Esa tercera comprobación fue un resultado engañoso: la ruta inicial /index.php no apuntaba al archivo público; una respuesta de error tampoco contiene esa cadena. La descarga de /app/datos.php demuestra que el sistema inicial seguía expuesto.



Figura 3. Después de corregir root y FastCGI app/datos.php devolvió 404 e index.php se ejecutó

La aplicación quedó fuera de la raíz pública. /app/datos.php ahora se busca dentro de public/app/, donde no existe. El código real queda en el directorio hermano app/ y PHP lo carga internamente mediante require.

Parte B Configuración completa de Nginx

Se completaron los cuatro TODO: raíz pública e índice PHP, FastCGI, proxy HTTP con cabeceras y página de mantenimiento para 502, 503 y 504.

```
server {
    listen 80;
    listen [::]:80;
    server_name portal.horizonte.internal;
    root /srv/horizonte/portal/public;
    index index.php;
    access_log /var/log/nginx/horizonte-portal.access.log horizonte;
    error_log /var/log/nginx/horizonte-portal.error.log warn;
    client_max_body_size 2m;
    location / { try_files $uri $uri/ =404; }
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
    location /api/ {
        proxy_pass http://127.0.0.1:8081/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    location ~ /\. { deny all; }
    error_page 502 503 504 /50x.html;
    location = /50x.html {
        root /srv/horizonte/www;
        internal;
    }
}
```

snippets/fastcgi-php.conf incluye la comprobación de existencia y los parámetros FastCGI de Ubuntu. fastcgi_pass usa el socket local de PHP 8.3. La página 50x.html pertenece al sitio institucional y solo se sirve como error por la directiva internal.

Parte B Sesión segura y servicio de la API

Cambios de la sesión y del inicio de sesión

```
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'secure' => ($_SERVER['HTTPS'] ?? '') === 'on',
    'httponly' => true,
    'samesite' => 'Lax',
]);
// Después de validar CSRF y contraseña y antes de asignar usuario
session_regenerate_id(true);
```

HttpOnly impide leer esta cookie desde JavaScript. SameSite=Lax restringe su envío en contextos entre sitios, especialmente POST externos, pero no reemplaza la validación CSRF. La regeneración cambia el ID al autenticar y elimina el archivo anterior, reduciendo la fijación de sesión. La comprobación strict_mode ya venía activada. [3]

Unidad horizonte eco completada

```
[Unit]
Description=API de eco de la U.E. Horizonte detras de Nginx
After=network.target
[Service]
Type=simple
User=www-data
Group=www-data
Environment=PUERTO_ECO=8081
Environment=PYTHONUNBUFFERED=1
ExecStart=/usr/bin/python3 /opt/horizonte/servidor_eco.py
Restart=on-failure
RestartSec=2
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
[Install]
WantedBy=multi-user.target
```

Los tres TODO quedaron resueltos con User y Group, ExecStart absoluto, y Restart con RestartSec. systemd-analyze verify aceptó la unidad. ProtectSystem, ProtectHome y PrivateTmp pertenecen al aislamiento de systemd: el adaptador de Docker no implementa esos espacios de aislamiento. Dentro del contenedor el código de la API pertenece a root y www-data no puede modificarlo.

Parte B Instalación tras cada cambio y verificación final

Se ejecutaron instalar_portal.sh y verificar_lab04.sh tras cada uno de los diez cambios, usando una copia inicial con TODO y conservando la configuración final al terminar.

Cambio	Configuración añadida	Resultado
1	Root public e index PHP	3 de 16
2	FastCGI hacia PHP FPM	11 de 16
3	Proxy API y cabeceras	11 de 16
4	Pagina 502 interna	11 de 16
5	Cookie HttpOnly	11 de 16
6	Cookie SameSite Lax	13 de 16
7	Regeneracion de sesion	14 de 16
8	Usuario y grupo www-data	14 de 16
9	ExecStart completo	16 de 16
10	Reinicio al fallar y espera	16 de 16

El resultado alcanzó 16 antes del último TODO porque el verificador no comprueba RestartSec ni todas las opciones de aislamiento. Se completó también ese requisito y se probó el reinicio real de la API. Los resultados intermedios son los observados durante las instalaciones; las comprobaciones funcionales se repitieron con la configuración final.

Parte B Evidencia de las dieciséis comprobaciones



Figura 4. Verificación final del script original con 16 de 16

El script confirmó ejecución de PHP, acceso privado bloqueado, atributos de la cookie, inicio de sesión, notas, regeneración del ID, token CSRF, cierre de sesión y proxy hacia la API.

Parte C Inicio de sesión desde el navegador

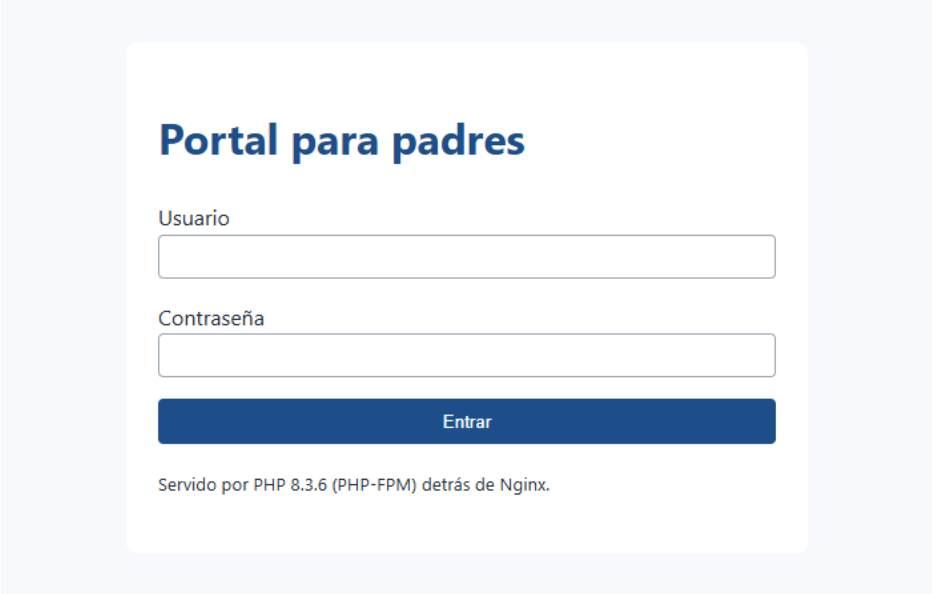


Figura 5. Formulario del portal PHP en http://portal.horizonte.internal:8080/



Figura 6. Panel autenticado con Rosa Mamani Lucas Mamani y las notas

La cuenta mamani.rosa permitió entrar al panel. Se mostraron Matemáticas 78, Física 85, Lenguaje 91 y Computación 98. Antes de entrar, el ID comenzaba por con5ku; después de validar la contraseña comenzó por a40tvu. La interfaz de depuración confirmó que el ID cambió.

Parte C Cookie y almacenamiento de la sesión

Dato de la cookie	Valor observado
Nombre	HORIZONTESESSID
Primeros seis caracteres	a40tvu
Dominio y ruta	portal.horizonte.internal y /
HttpOnly	true
SameSite	Lax
Secure	false porque esta práctica usa HTTP local
Vencimiento	De sesión; expires = -1 en la interfaz del navegador



Figura 7. Atributos reales de la cookie y confirmación de regeneración mediante la interfaz de depuración

La cookie contiene únicamente un identificador opaco. El nombre de Rosa Mamani y el usuario no se guardan en esa cookie. El servidor asocia el ID con un archivo sess_<ID>. HttpOnly no evita que el navegador envíe la cookie; limita el acceso desde JavaScript. Secure se activaría al servir por HTTPS. [4]

La hoja indica revisar Application y Cookies en DevTools. En esta ejecución se consultaron los mismos atributos almacenados usando la interfaz de depuración del navegador; la figura corresponde al registro de esa consulta.

Parte C Archivo del servidor y cierre de sesión

```
ls -l /var/lib/php/sessions
cat /var/lib/php/sessions/sess_a40tvua707lametikmq0lugqop
```

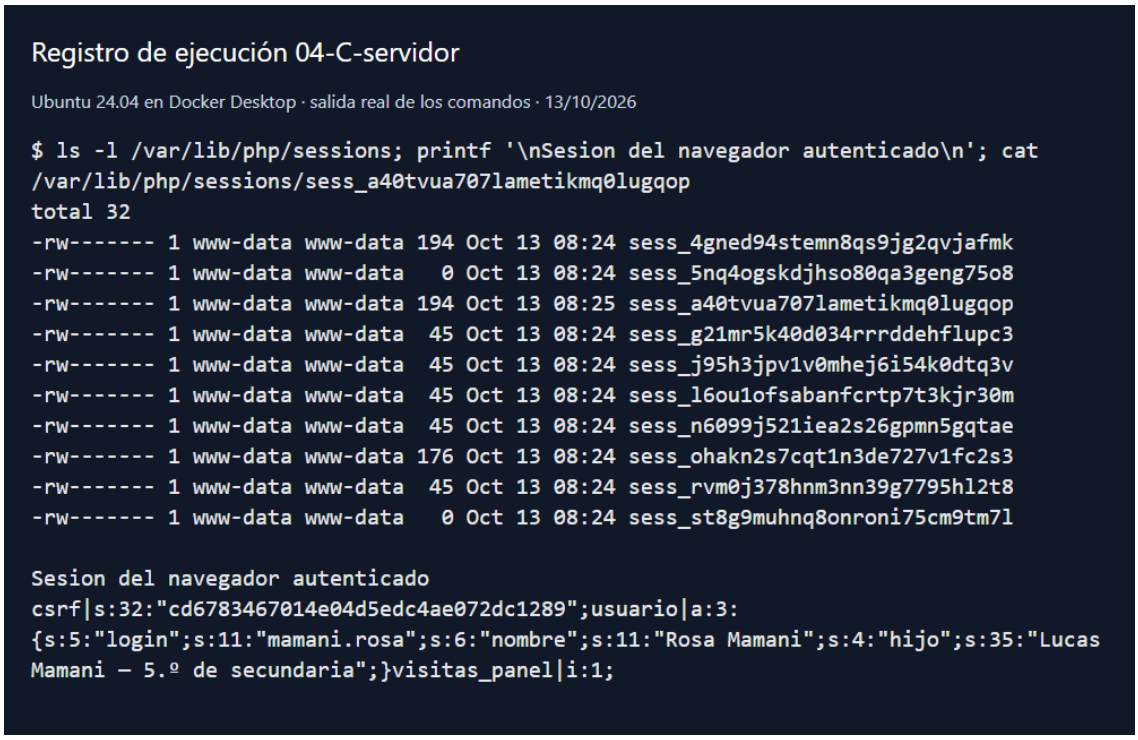


Figura 8. Archivo de la sesión autenticada con datos serializados de PHP

El archivo tuvo modo 600 y dueño www-data. Guardó csrf, el arreglo usuario con login mamani.rosa, nombre Rosa Mamani e hijo Lucas Mamani, además de visitas_panel con valor 1. El nombre y los datos de la sesión estaban en el servidor, asociados al ID del navegador.

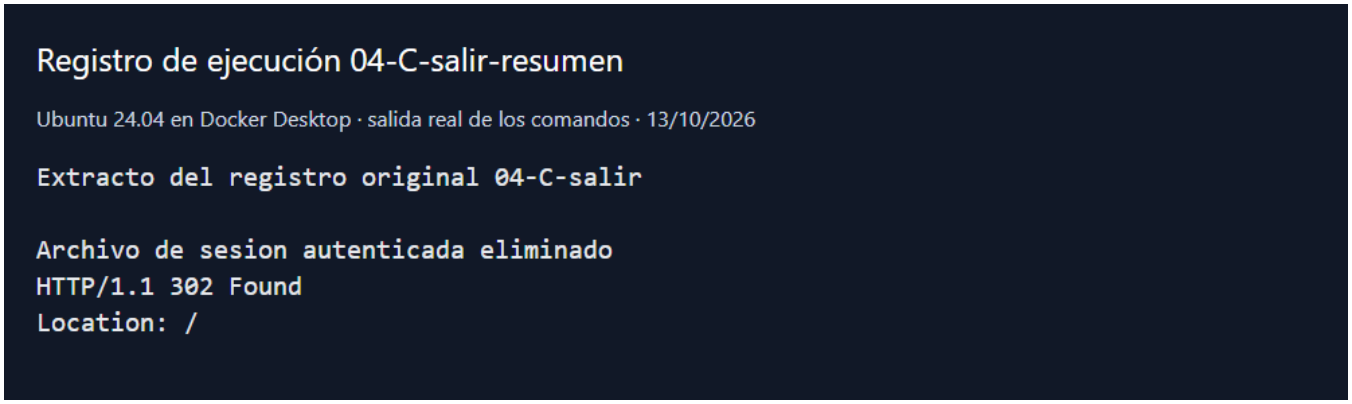


Figura 9. Después de cerrar sesión el archivo autenticado se eliminó y el panel sin sesión devolvió 302

salir.php vació la sesión, venció la cookie y ejecutó session_destroy(). Al seguir la redirección hacia el formulario se creó una nueva sesión anónima para el token CSRF. Por eso puede aparecer otro archivo y otra cookie después de salir; no es la sesión autenticada que se eliminó.

Parte D Proxy HTTP y cabeceras reenviadas

```
curl -H "Host: portal.horizonte.internal" \
  -H "X-Forwarded-For: 203.0.113.9" http://127.0.0.1/api/eco
```



Figura 10. Respuesta JSON de la API con la IP de conexión y las cabeceras del proxy

Campo	Valor y origen
cliente	127.0.0.1. La API recibió la conexión de Nginx desde el mismo contenedor.
X-Forwarded-For	203.0.113.9, 127.0.0.1. El cliente envió el primer valor y Nginx añadió remote_addr.
X-Real-IP	127.0.0.1. Lo fijó Nginx con $remote_addr.
Host y X-Forwarded-Proto	portal.horizonte.internal y http, establecidos por la configuración de Nginx.
ruta y version_http	/eco y HTTP/1.0 en el salto interno del proxy; no son la URL ni la versión del navegador.

La barra final de proxy_pass http://127.0.0.1:8081/ reemplaza el prefijo /api/ por /: /api/eco llega como /eco. Sin esa URI final, se reenviaría /api/eco y este servidor respondería 404. Un X-Forwarded-For recibido de un cliente no es por sí solo una prueba de su IP real; aquí el primer valor se introdujo deliberadamente. [5]

Parte D Procesos PHP socket y ejecución sin privilegios

```
ps -o user,pid,cmd -C php-fpm8.3
ls -l /run/php/
```

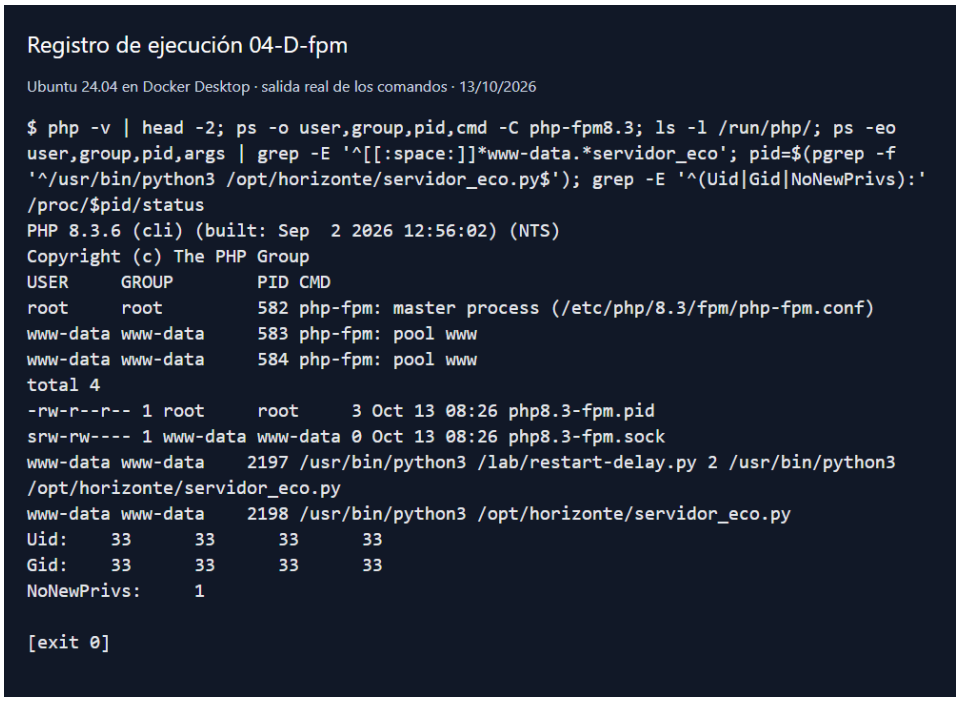


Figura 11. Procesos PHP socket Unix y usuario grupo y NoNewPrivs de la API

El proceso maestro de PHP-FPM fue root y los trabajadores del pool www fueron www-data. Los trabajadores ejecutan las peticiones PHP. /run/php/php8.3-fpm.sock fue un socket Unix, identificado por la s al inicio de los permisos; es un canal local entre Nginx y PHP-FPM, no un archivo PHP ni un puerto publicado en Windows.

La API se ejecutó con UID y GID 33, correspondientes a www-data, y NoNewPrivs: 1. El supervisor realiza la reducción de usuario y grupo antes de lanzar Python. No se usó el modo privilegiado de Docker.



Figura 12. Tras terminar el proceso de la API el supervisor lo reinició con otro PID y respondió 200

Se provocó un fallo del proceso de la API con SIGTERM y se comprobó un PID diferente cinco segundos después. El adaptador espera dos segundos tras una salida anormal antes de que Supervisor vuelva a iniciarlo. El verificador original volvió a dar 16 de 16.

Parte E Caída controlada de PHP FPM

```
systemctl stop php8.3-fpm
curl -I -H "Host: portal.horizonte.internal" http://127.0.0.1/
tail -1 /var/log/nginx/horizonte-portal.error.log
systemctl start php8.3-fpm
```

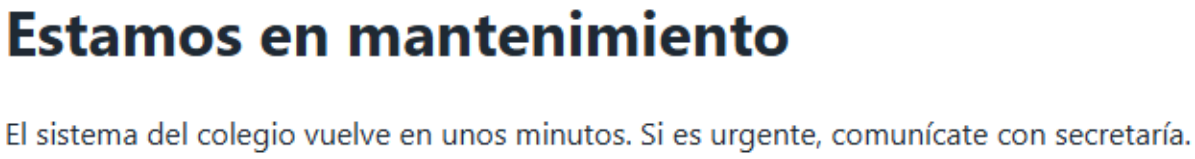


Figura 13. Portal con PHP detenido el padre ve Estamos en mantenimiento y la respuesta fue 502



Figura 14. Cabeceras 502 mensaje del socket inexistente y posterior inicio de PHP

El mensaje fue connect() to unix:/run/php/php8.3-fpm.sock failed (2: No such file or directory) while connecting to upstream. PHP había dejado de ofrecer el socket y Nginx no pudo ejecutar la petición. El cuerpo de mantenimiento sustituyó el error técnico, pero mantuvo HTTP/1.1 502 Bad Gateway. Después se restauró PHP-FPM.

Parte E Caída de la API y contraseña incorrecta

```
systemctl stop horizonte-eco
curl -I -H "Host: portal.horizonte.internal" http://127.0.0.1/api/eco
tail -1 /var/log/nginx/horizonte-portal.error.log
systemctl start horizonte-eco
```

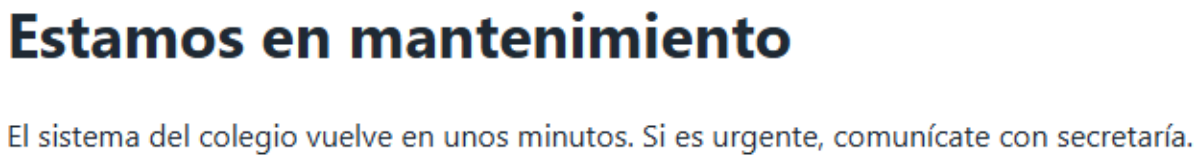


Figura 15. Con la API detenida /api/eco mostró la página de mantenimiento con código 502



Figura 16. La conexión HTTP fue rechazada al detener la API y volvió a 200 al restaurarla

El error fue connect() failed (111: Connection refused) while connecting to upstream, con upstream http://127.0.0.1:8081/eco. En este caso faltó el servidor HTTP interno, mientras Nginx siguió atendiendo. Los procesos de la API se detuvieron como grupo para que no quedara un proceso hijo ocupando el puerto.

Parte E Acceso fallido y reflexión final



Figura 17. Contraseña incorrecta el portal mostró el mensaje de acceso fallido

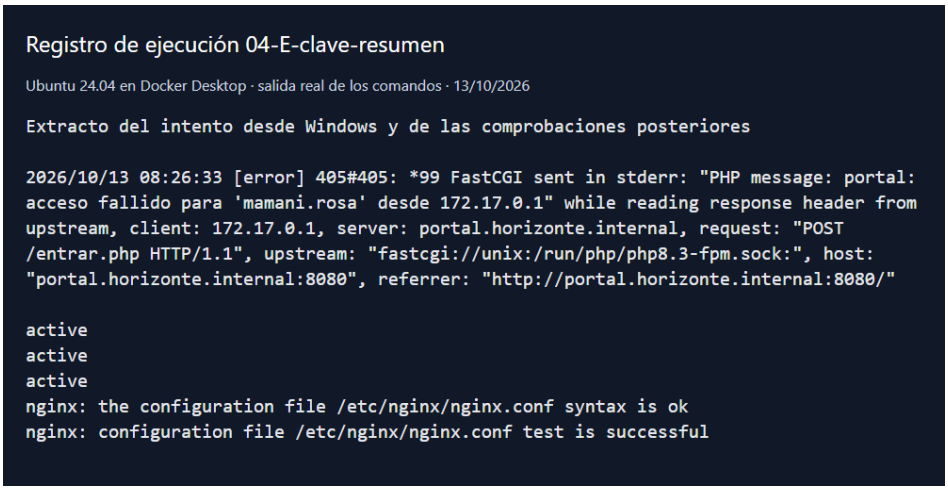


Figura 18. Registro del intento desde Windows y estados de servicio tras las pruebas

El POST con la contraseña incorrecta volvió con 303 a /?error=1. El error.log registró portal: acceso fallido para mamani.rosa desde 172.17.0.1, la dirección que Nginx vio para la conexión desde Windows. El verificador también comprobó el rechazo de tokens CSRF incorrectos con 403 y el panel sin sesión con redirección 302.

FastCGI frente a proxy inverso HTTP

FastCGI pasa parámetros y la petición a PHP-FPM, que ejecuta el archivo PHP y devuelve la salida. El proxy HTTP envía una petición HTTP a otro servidor que ya conoce sus rutas. En este laboratorio FastCGI usa un socket Unix y la API usa TCP en 127.0.0.1:8081. Ambas rutas necesitan un backend activo, pero sus protocolos y diagnósticos de caída son diferentes.

Por qué separar app de public

Nginx debe poder entregar solo los recursos destinados al navegador. El código interno, datos y utilidades de sesión permanecen fuera de public/; así un error al desactivar PHP no permite descargar esos archivos privados directamente. La prueba inicial mostró el riesgo y la final confirmó 404 para /app/datos.php.

Fuentes y cierre de la práctica

Se completaron las partes A a E de la hoja y sus preguntas, los diez TODO y las verificaciones de sesiones, proxy, procesos y errores. Las pruebas finales devolvieron 16 de 16 después de restaurar los servicios. La ejecución fue asistida con ChatGPT y los resultados se contrastaron con las salidas reales y la documentación oficial.

[3] PHP regeneración del identificador de sesión
https://www.php.net/manual/en/function.session-regenerate-id.php

[4] PHP configuración de sesiones y cookies
https://www.php.net/manual/en/session.configuration.php

[5] Nginx proxy_pass y cabeceras del proxy
https://nginx.org/en/docs/http/ngx_http_proxy_module.html#proxy_pass

Reproducción en el entorno Docker preparado

```
docker start horizonte-labs-20261013
docker exec horizonte-labs-20261013 systemctl start php8.3-fpm
docker exec horizonte-labs-20261013 systemctl start horizonte-eco
docker exec horizonte-labs-20261013 bash /lab/lab04/verificar_lab04.sh
```

Dentro del contenedor se usan las direcciones 127.0.0.1 sin :8080. Desde Windows se usa :8080 y Host portal.horizonte.internal. La regla de resolución aplicada durante las capturas perteneció al navegador de prueba y terminó al cerrarlo. Para comprobar el portal desde Windows sin cambiar hosts puede enviarse la cabecera Host con curl.exe.

```
curl.exe -I -H "Host: portal.horizonte.internal" http://127.0.0.1:8080/
```